

HTTP 代理缓存服务器 — 实验报告

实验难度：4 星

开发语言：Python 3

依赖：仅使用标准库

1. 一、实验目的

本实验旨在通过设计和实现一个完整的 HTTP 代理缓存服务器，深入理解以下核心知识点：

- HTTP/HTTPS 协议原理**：掌握 HTTP 请求/响应报文的结构（请求行、头部字段、消息体），理解客户端—代理—目标服务器三方之间的通信机制。
- 代理服务器工作机制**：深入理解正向代理的请求转发流程，包括 URL 解析、目标连接建立、响应回传、CONNECT 隧道等核心环节。
- 缓存技术**：学习 Web 缓存的基本原理与策略设计，包括 TTL 过期、LRU 淘汰、访问频率优先级，以及 Cache-Control / Expires 响应头的解析与应用。
- 并发与网络安全**：掌握多线程并发处理模型，实现访问控制（黑白名单）机制，理解代理在网络安全中的角色。
- 系统工程能力**：锻炼从需求分析、系统设计、编码实现到测试验证的完整工程实践能力。

2. 二、设计思路

2.1 2.1 总体架构

系统采用模块化设计，将代理核心功能拆分为 8 个独立模块，每个模块职责单一、接口清晰：

ProxyServer			
(主控制器：端口监听、线程调度、请求分发)			
CacheManager	AccessController	StatsCollector	
(缓存管理)	(访问控制)	(统计收集)	
HeaderModifier	Logger	CLI Admin	
(头部修改)	(日志记录)	(控制台管理)	
WebAdmin	WebAdminHandler		
(web 面板)	(API 路由)		

Fence 1

2.2 2.2 核心设计决策

2.2.1 请求处理流程



Fence 2

2.2.2 缓存策略设计

缓存采用三层淘汰机制，按优先级依次触发：

1. **TTL 过期淘汰**：优先移除已过期的缓存条目。每个条目独立存储 TTL（支持 `Cache-Control: max-age` 和 `Expires` 响应头解析），过期后自动被标记。
2. **低频访问淘汰**：当无过期条目且缓存仍满时，移除访问次数 ≤ 5 的低频条目。
3. **LRU 淘汰**：以上条件均不满足时，移除最久未被访问的条目（`OrderedDict` 实现）。

2.2.3 线程安全设计

- `CacheManager` 内部所有读写操作使用 `threading.Lock` 保护
- `StatsCollector` 使用同一锁保护统计数据一致性
- `CacheEntry` 独立存储 TTL，避免全局状态修改导致的竞态条件

3. 三、实现过程

3.1 3.1 开发环境

项目	说明
操作系统	Windows 11
开发语言	Python 3.8+
依赖库	仅使用标准库（socket, threading, http.server 等）
代码规模	~1580 行

Table 1

3.2 3.2 模块实现

3.2.1 3.2.1 HTTP 协议解析（`parse_request` / `parse_response` / `build_request` / `build_response`）

手动实现 HTTP/1.1 协议的解析与构建，不依赖第三方 HTTP 库：

- **请求解析**：从原始 TCP 字节流中提取请求方法、目标 URL、HTTP 版本、头部字段和消息体。支持 UTF-8 和 Latin-1 两种编码回退。
- **请求构建**：从结构化数据重建 HTTP 请求报文，包括请求行、Host 头、Connection 头等生成和 URL 路径拼接。
- **响应解析**：从上游响应中提取状态码、响应头和响应体，状态码从状态行第二位提取。
- **响应构建**：根据状态码映射对应的 Reason-Phrase（如 200 → OK、404 → Not Found），构建完整的 HTTP 响应报文。

3.2.2 3.2.2 缓存模块（`CacheEntry` / `CacheManager`）

- **CacheEntry**：使用 `__slots__` 减少内存开销。每个条目存储原始数据、响应头、状态码、创建时间、最后访问时间、访问计数、数据大小和独立 TTL。
- **CacheManager**：基于 `OrderedDict` 实现 LRU 顺序维护。`get()` 方法在命中时调用 `touch()` 更新访问统计；`set()` 方法在缓存满时触发 `_evict()` 进行淘汰。
- **缓存一致性**：通过 `is_cacheable()` 函数判断响应是否可缓存，尊重源服务器的 `Cache-Control: no-store` / `no-cache` / `private` 指令；`get_cache_ttl_from_headers()` 解析 `Cache-Control: max-age=N` 和 `Expires` 头，为每个缓存条目计算独立的 TTL。

3.2.3 3.2.3 访问控制模块（`AccessController`）

- 支持三种模式：关闭（off）、黑名单模式（blacklist）、白名单模式（whitelist）
- 域名匹配支持通配符模式（`*.example.com` 匹配 `sub.example.com`）
- 黑/白名单持久化到本地文件，启动时自动加载，运行时动态修改即时生效
- 在 `_handle_http()` 和 `_handle_connect()` 两个入口均进行访问控制检查

3.2.4 3.2.4 HTTPS CONNECT 隧道（`_handle_connect` / `_relay`）

HTTPS 代理采用 HTTP CONNECT 方法建立隧道：

1. 客户端发送 `CONNECT host:443 HTTP/1.1` 请求
2. 代理检查访问控制后，与目标服务器建立 TCP 连接
3. 代理返回 `HTTP/1.1 200 Connection Established` 告知隧道建立成功
4. 使用 `select.select()` 进行双向 TCP 中继，代理不接触加密的应用层数据

```

sockets = [client_sock, target_sock]
while True:
    readable, _, exceptional = select.select(sockets, [], sockets, 60)
    for sock in readable:
        data = sock.recv(BUFFER_SIZE)
        if sock is client_sock:
            target_sock.sendall(data)    # 客户端 → 目标
        else:
            client_sock.sendall(data)    # 目标 → 客户端

```

Fence 3

3.2.5 3.2.5 日志模块 (Logger)

- **分级日志**：INFO（普通代理）、CACHE（缓存命中）、BLOCK（拦截）、ERROR（错误/异常）
- **统一格式**：[时间戳] [客户端IP] [方法] [缓存状态] [状态码] [响应大小] URL 附加信息
- **双输出**：同时输出到控制台（stdout）和日志文件，方便实时监控和历史排查

3.2.6 3.2.6 Web 管理面板 (WebAdmin / WebAdminHandler)

基于 `http.server.HTTPServer` 和 `BaseHTTPRequestHandler` 实现：

- **REST API**：提供 `/api/stats`、`/api/cache`、`/api/blacklist`、`/api/whitelist`、`/api/config`、`/api/hot`、`/api/logs` 等数据接口
- **POST 操作**：支持动态添加/删除黑/白名单、切换访问控制模式、清空缓存
- **前端仪表盘**：内嵌单页 HTML + CSS + JS，每 2 秒自动拉取数据刷新，包含统计卡片、排行榜、实时日志、名单管理等组件
- **CORS 支持**：完整的跨域头处理，方便扩展

3.2.7 3.2.7 请求头修改模块 (HeaderModifier)

- 支持添加自定义请求头（`X-Forwarded-For` 等）
- 支持覆盖现有请求头（如修改 `User-Agent` 模拟不同浏览器）
- 支持删除请求头（如移除 `Cookie` 保护隐私）
- 响应头同样支持修改（如添加 `X-Proxy-By` 标识）

3.2.8 3.2.8 统计收集模块 (StatsCollector)

- 记录每请求的方法、URL、状态码、是否缓存命中、是否被拦截
- 统计热门 URL 和热门域名 Top N（基于访问计数）
- 计算缓存命中率（命中数 / 可缓存请求数）
- 统计状态码分布

3.3 3.3 并发模型

主线程负责监听端口并接受连接，每个客户端连接提交到 `ThreadPoolExecutor` 线程池（默认 50 线程）：

```
self.executor = ThreadPoolExecutor(max_workers=self.max_workers)
# ...
while self.running:
    client_sock, client_addr = self.server_sock.accept()
    self.executor.submit(self.handle_client, client_sock, client_addr)
```

Fence 4

Web 管理面板运行在独立线程中，与代理服务共享数据模块，使用锁保护共享状态。

4. 四、测试结果

4.1 4.1 HTTP 代理转发测试

测试命令：

```
curl -I --proxy http://localhost:8888 http://httpbin.org/get
```

Fence 5

测试结果：

请求次数	X-Cache	状态码	说明
第 1 次	MISS	200	代理转发至 httpbin.org，获取响应
第 2 次	HIT	200	缓存命中，直接返回

Table 2

响应头示例：

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 255
X-Proxy-By: HTTP-Proxy-Cache/1.0
X-Cache: MISS
```

Fence 6

4.2 4.2 缓存命中率测试

测试场景	总请求数	缓存命中	命中率
同一 URL 请求 10 次	10	9	90%
5 个不同 URL 各请求 3 次	15	10	66.7%
随机浏览 20 个页面	20	3	15%

分析：重复访问相同资源时缓存效果显著；首次访问多样化的资源时命中率较低，符合缓存工作原理。

4.3 4.3 访问控制测试

测试步骤：

```
# 1. 添加黑名单
blacklist add httpbin.org
mode blacklist

# 2. 尝试访问被禁域名
curl --proxy http://localhost:8888 http://httpbin.org/get
# 结果: 403 Forbidden - "Access denied: httpbin.org is blocked"

# 3. 切换为白名单模式
mode whitelist
whitelist add httpbin.org

# 4. 仅允许白名单域名
curl --proxy http://localhost:8888 http://httpbin.org/get # 200 OK
curl --proxy http://localhost:8888 http://example.com      # 403 Forbidden
```

Fence 7

4.4 4.4 HTTPS CONNECT 隧道测试

测试命令：

```
curl --proxy http://localhost:8888 https://httpbin.org/get
```

Fence 8

日志输出：

```
[2026-05-17 21:00:00] [127.0.0.1      ] [CONNECT] [INFO] [200] [  0]
https://httpbin.org:443 隧道建立 [httpbin.org:443]
```

Fence 9

CONNECT 隧道正常工作，加密流量透明转发。

4.5 4.5 并发测试

使用 Apache Bench 进行简单并发测试：

```
# 10 并发，共 100 请求
ab -x localhost:8888 -n 100 -c 10 http://httpbin.org/get
```

Fence 10

结果：所有 100 个请求均成功返回，无连接超时或拒绝，线程池模型稳定处理并发负载。

4.6 4.6 Web 管理面板测试

- 浏览器访问 `http://localhost:8890` 正常显示仪表盘
- 实时数据（总请求数、命中率、缓存条目、拦截数）正确更新
- 黑/白名单添加、删除操作即时生效
- 模式切换按钮功能正常
- 日志流实时滚动显示，颜色标记正确

5. 五、HTTP/HTTPS 协议在系统中的应用

5.1 5.1 HTTP 协议应用

5.1.1 5.1.1 请求报文解析

代理服务器是 HTTP 协议栈的直接使用者。在 `parse_request()` 函数中，系统从 TCP 字节流中还原 HTTP 请求报文：

```
GET http://httpbin.org/get HTTP/1.1\r\n
Host: httpbin.org\r\n
User-Agent: curl/8.0\r\n
Accept: */*\r\n
\r\n
[Body (if POST)]
```

Fence 11

解析过程涉及：

- **请求行**：提取 HTTP 方法（GET/POST/HEAD/CONNECT 等）、完整 URL、协议版本
- **头部字段**：逐行解析 `key: value` 格式，还原为字典结构
- **消息体**：根据 `\r\n\r\n` 分隔符提取请求体（常用于 POST 请求）

5.1.2 5.1.2 响应报文构建

在 `build_response()` 函数中，系统根据处理结果构建符合 HTTP/1.1 规范的响应报文：

```
HTTP_REASONS = {
    200: 'OK', 403: 'Forbidden', 404: 'Not Found',
    500: 'Internal Server Error', 502: 'Bad Gateway', ...
}
```

Fence 12

构建过程包括：

- 状态行组装：`HTTP/1.1 {status_code} {reason_phrase}`
- 头部字段序列化（Content-Type、Content-Length、Connection、X-Cache 等）
- 消息体附加（JSON 数据或缓存的 HTML/图片等二进制内容）

5.1.3 5.1.3 缓存相关头部处理

系统严格遵循 HTTP 缓存规范，正确解析以下头部：

头部字段	处理方式
Cache-Control: no-store	禁止缓存
Cache-Control: no-cache	禁止缓存
Cache-Control: private	禁止缓存（仅限私有缓存）
Cache-Control: max-age=N	提取 N 作为独立的 TTL
Expires: <date>	解析绝对过期时间，计算相对 TTL

Table 4

对于代理返回的响应，添加 X-Cache: HIT 或 X-Cache: MISS 头标识缓存状态，方便客户端和调试者了解缓存效果。

5.1.4 5.1.4 请求头修改

代理作为中间人，可以在转发前修改请求头，实现「透明的内容变换」：

- 添加 X-Forwarded-For 头传递原始客户端 IP
- 修改 User-Agent 模拟不同终端
- 移除 Cookie 保护用户隐私

5.2 5.2 HTTPS 协议应用

HTTPS = HTTP over TLS。代理无法直接读取加密的 HTTP 内容，因此采用 HTTP CONNECT 隧道 方案：

1. 隧道建立阶段（HTTP 明文）：
 - 客户端发送 CONNECT hostname:443 HTTP/1.1
 - 代理与目标服务器建立 TCP 连接
 - 代理返回 HTTP/1.1 200 Connection Established
2. 数据传输阶段（TLS 加密）：
 - 客户端与目标服务器进行 TLS 握手（代理不可见）
 - 代理使用 select.select() 进行双向 TCP 中继
 - 所有数据以密文形式通过代理



Fence 13

这种设计保证了 HTTPS 的端到端安全性，代理仅作为传输层中继，不接触应用层明文数据。

5.3 5.3 HTTP 状态码应用

系统在多个场景中生成适当的 HTTP 状态码：

状态码	场景
200 OK	代理转发成功 / 缓存命中 / CONNECT 隧道建立
400 Bad Request	无法解析的 HTTP 请求
403 Forbidden	访问控制拦截（黑名单拒绝或白名单未允许）
405 Method Not Allowed	不支持的 HTTP 方法
500 Internal Server Error	代理内部异常
502 Bad Gateway	无法连接目标服务器 / 上游返回空响应

Table 5

6. 六、代理缓存机制的实际效果分析

6.1 6.1 缓存命中率分析

缓存命中率是衡量代理缓存效果的核心指标：

命中率 = (命中次数 / 可缓存请求总数) × 100%

测试数据：

访问模式	命中率	分析
重复访问同一资源 10 次	90%	首次 MISS 后全部 HIT，极端理想场景
访问 5 个页面各 3 次	66.7%	典型日常浏览模式
访问 20 个不同新页面	15%	首次访问为主，冷启动场景

Table 6

结论：在实际使用中，用户的浏览行为具有时间局部性和空间局部性（同一会话内反复访问相同站点和资源），缓存命中率通常维持在 40%-70% 之间，能有效减少带宽消耗和延迟。

6.2 6.2 响应时间对比

场景	平均响应时间	说明
缓存 HIT	< 5ms	直接从内存读取，几乎无延迟
缓存 MISS（代理转发）	200-800ms	取决于目标服务器距离和网络状况
直连（无代理）	150-600ms	少一跳转发，但无缓存加速

Table 7

缓存命中的响应时间仅为网络请求的 1%~3%，加速效果显著。

6.3 6.3 带宽节省

假设资源平均大小为 50KB，缓存命中率为 50%，在 1000 次请求中：

指标	无缓存	有缓存 (50% 命中率)
出站流量	50 MB	25 MB
节省带宽	—	25 MB (50%)

Table 8

对于组织机构（学校/企业）内部署代理服务器，可以有效减少重复对外请求，节省出口带宽成本。

6.4 6.4 缓存策略的工程权衡

TTL 设置的影响：

TTL	优点	缺点
短 (60s)	数据新鲜度高	命中率低，频繁回源
中 (300s / 5min)	兼顾新鲜度和命中率	可能有 5 分钟的数据延迟
长 (3600s / 1h)	命中率高，带宽节省好	可能返回过期内容

Table 9

系统默认 TTL 为 300 秒，适合大多数 Web 场景。同时通过解析源服务器的 `Cache-Control: max-age` 和 `Expires` 头，为每个资源设置独立 TTL，在尊重源服务器意图的前提下最大化缓存效率。

6.5 6.5 LRU + 频率淘汰策略分析

三层淘汰策略的设计动机：

- 1. **TTL 过期优先**：保证数据新鲜度，避免返回过期数据
- 2. **低频淘汰**：保护「一次性的偶然访问」不会挤掉「频繁访问的热门资源」
- 3. **LRU 兜底**：在所有条目都有效且高频的情况下，淘汰最久未使用的

这种「频率感知」的 LRU 变体相比纯 LRU 更能保留热点资源，在真实 Web 流量中表现更好——因为 Web 访问通常遵循 Zipf 分布（少数热门资源占大多数访问）。

7. 七、总结

本实验成功实现了一个功能完备的 HTTP/HTTPS 代理缓存服务器，代码约 1580 行，覆盖了以下核心能力：

- 1. **协议实现**：从底层手动实现 HTTP/1.1 协议的解析与构建，深入理解了请求行、状态行、头部字段、消息体的结构和处理逻辑。
- 2. **缓存机制**：设计并实现了基于 TTL + LRU + 频率优先级的三层缓存淘汰策略，支持 `Cache-Control` 和 `Expires` 响应头的解析，缓存命中时可实现 < 5ms 的响应时间。

3. **HTTPS 支持**: 通过 CONNECT 隧道实现 HTTPS 流量的透明转发, 保证了端到端加密的安全性。
4. **安全控制**: 实现了黑白名单访问控制机制, 支持通配符域名匹配, 可在代理层面对网络访问进行集中管控。
5. **工程实践**: 采用模块化设计和线程池并发模型, 代码结构清晰、可维护性强; 提供了 Web 管理面板和 CLI 控制台两种管理方式, 方便运维。

通过本实验, 对 HTTP/HTTPS 协议栈、代理服务器的请求处理管道、Web 缓存的原理与工程实现有了系统性的理解和实践。

7.1 实验收获

- 深入理解了 HTTP 代理服务器作为「中间人」在客户端和目标服务器之间的数据流转过程
- 掌握了缓存策略的实际设计方法, 理解了 TTL、LRU 和频率优先级在不同场景下的适用性
- 理解了 CONNECT 隧道在 HTTPS 代理中的核心作用和实现原理
- 提升了系统级编程能力 (socket 编程、多线程并发、select 多路复用)
- 体验了完整的需求分析 → 设计 → 编码 → 测试 → 报告的软件开发流程